

Campus Event Manager

NoSQL Development Project — MCS DE1, Master 1 Data Engineering and Cloud Computing

Author: **Vivek Bhalani** · Database: MongoDB 8 · Date: 12 September 2026

Repository: github.com/vivek-bhalani-ai/campus-event-manager

1. Project overview

Campus Event Manager is a small web application that lets a university manage campus events (workshops, talks, hackathons, networking sessions and social activities), the people attached to them, and the registrations that link the two. The database did not exist at the start of the project: the collections, their validation rules, their indexes and their seed data are all created from scripts committed to the repository.

The objective was not to build a complete university platform but to exercise a full document-oriented stack end to end: model the data as documents rather than as tables, write the CRUD and aggregation operations directly against the driver, and expose the result through five working pages.

Technology

Layer	Choice and reason
Database	MongoDB 8, run locally (mongodb://localhost:27017). An Atlas free cluster was configured first, but the working network blocked outbound port 27017, so the project runs against a local server. The connection string is the only thing that differs between the two.
Driver	PyMongo. Deliberately no ODM: every filter, update operator and pipeline stage stays visible in the source instead of being hidden behind a model class.
Backend	Python 3.13 with Flask, organised into blueprints.
Frontend	Jinja2 server-side templates with a single hand-written stylesheet. No JavaScript framework: the brief asks for a usable interface, not a UI exercise.

2. NoSQL data model

The database `campus_events` holds two collections. A third collection for registrations was considered and rejected — see section 3.

users

```
{
  _id:      ObjectId,
  firstName: "Amelie",
  lastName:  "Rousseau",
  email:    "amelie.rousseau@campus.edu", // unique index
  department: "Data Engineering",
  role:      "student", // student | teacher | staff
  interests: ["machine learning", "python"], // array of strings
  createdAt: ISODate
}
```

events

```
{
  _id:      ObjectId,
  title:    "NoSQL Modeling Deep Dive",
  description: "...",
  category: "Workshop",
  tags:     ["nosql", "databases"], // array of strings
  startDate: ISODate,
  endDate:  ISODate,
  capacity: 25,
  location: { // embedded object
    building: "Innovation Center",
    room:     "B204",
    campus:   "Paris"
  },
  organizerId: ObjectId, // reference -> users._id
  registrations: [ // array of embedded documents
    {
      userId:      ObjectId, // reference -> users._id
      registeredAt: ISODate,
      status:      "confirmed" // confirmed | waiting | cancelled
    }
  ],
}
```

```
    createdAt: ISODate
  }
```

The model mixes the three document-database techniques on purpose: *embedding* for `location`, *arrays* for `tags` and `interests`, and *referencing by ObjectId* for `organizerId` and `registrations[].userId`. The registration array is a hybrid: the entry itself is embedded in the event, but the person it points to is referenced.

3. Modeling choices

Choice 1 — location is embedded, not referenced

A venue has no independent life in this application. It is never listed on its own, never edited outside an event, and never queried without its event. It is also bounded: three short fields that will not grow. Embedding it means the detail page reads one document and has everything it needs, with no join and no second round trip.

The cost is duplication — "Innovation Center, B204" is repeated across events, and renaming a building would require an `updateMany`. That is an acceptable trade because campus rooms are renamed very rarely, whereas event pages are read constantly. Had rooms carried bookings, equipment and their own availability calendar, the decision would flip to a separate collection.

Choice 2 — registrations are embedded, but the user is referenced

A relational design would produce a join table `registrations(event_id, user_id, status)`. Mechanically reproducing that in MongoDB would be the wrong instinct here. Registrations are always read in the context of one event, they are bounded by `capacity` (tens, never thousands, so the 16 MB document limit is not a concern), and the participant list is written and read as a unit. Embedding keeps the whole event — its details, its occupancy and its participants — in a single document, and makes every registration operation a single atomic update.

The user, however, is *referenced* rather than copied. User data is mutable: an email change or a department move must not leave stale copies scattered across eighteen events. Storing only `userId` keeps one authoritative record per person, at the cost of a `$lookup` when a participant's name has to be displayed — a cost paid on one page, not on every write.

The resulting pattern: **embed what is owned and bounded** (location, registration entries); **reference what is shared and mutable** (users, organisers).

Validation and indexes

Validation exists at two levels. MongoDB JSON-Schema validators in `database/init.js` enforce the structure server-side, including a cross-field rule that JSON Schema alone cannot express:

```
validator: { $and: [
  { $jsonSchema: { /* types, required fields, enums, minLength, minimum */ } },
  { $expr: { $gte: ["$endDate", "$startDate"] } }
]}
```

The application repeats the same rules in `src/errors.py` so the user gets a readable message instead of a driver exception: empty title, capacity not greater than zero, end date before start date, invalid or duplicate email, and a registration status outside the controlled list.

Index	Supports
<code>users.email</code> (unique)	Mandatory. Guarantees one account per address; the driver's duplicate-key error is turned into a user-facing message.
<code>events.startDate</code>	Upcoming/past filters, date sorting, the dashboard's next-five query.
<code>events.category</code>	Category filter and the grouping in Analysis A.
<code>events.tags</code> (multikey)	Tag filter on the Events page and Analysis E.
<code>events.registrations.userId</code>	"Which events is this user in?" — user history and duplicate-registration checks.
<code>users.department + role</code>	Compound index for the Users page filters.

Seed data

`database/seed.js` clears both collections and reinserts a fixed dataset, so it can be re-run at any time and always produces the same database: **15 users**, **18 events**, **6 categories**, **21 tags** and **88 registration entries** (83 active once cancelled ones are excluded). The dataset is deliberately uneven — nine past and nine upcoming events, three events with no registration at all, several events at or near capacity, and start dates

spread from May to December 2026 so that the monthly grouping in Analysis F returns something meaningful. Every `organizerId` and every `registrations[].userId` is drawn from the inserted users, so no reference dangles.

4. Application architecture

```
campus-event-manager/
|-- database/
|   |-- init.js      validators + 6 indexes
|   |-- seed.js      reproducible dataset
|-- src/
|   |-- app.py        application factory, blueprint registration, error handlers
|   |-- db.py         single MongoClient, reads MONGODB_URI from .env
|   |-- errors.py     AppError + form validation mirroring the DB rules
|   |-- repositories/ ALL MongoDB operations (events, users, analytics)
|   |-- routes/       Flask blueprints: dashboard, events, users, analytics
|   |-- templates/    Jinja2 pages
|   |-- static/       stylesheet
|-- run.py
```

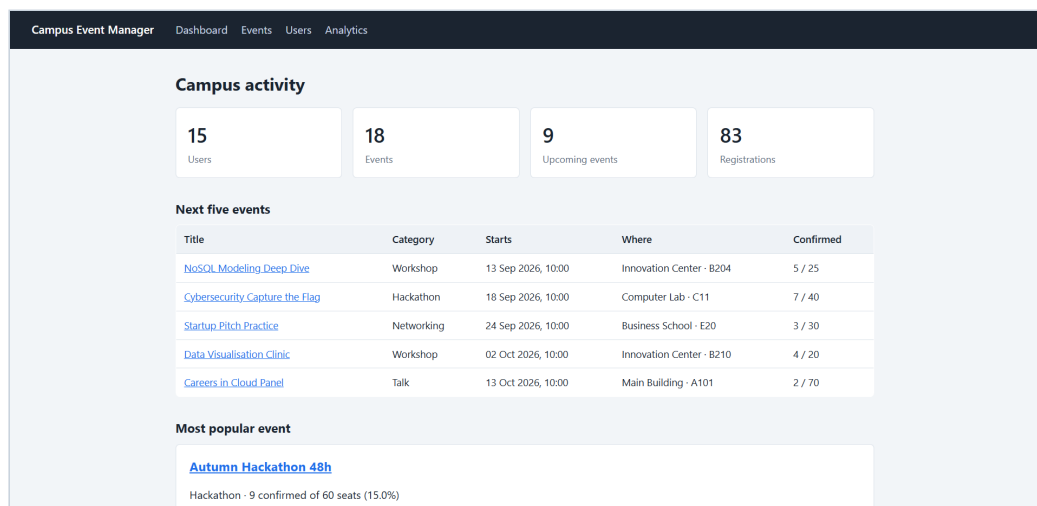
The layering is deliberate and is what makes the MongoDB work reviewable. Routes never touch the driver: they validate input, call a repository function and render a template. Every query, update operator and pipeline lives in `src/repositories/`, so the whole database surface of the project can be read in three files.

The connection is created once in `db.py` and reused for the life of the process. The URI comes from a `.env` file loaded with `python-dotenv`; `.env` is listed in `.gitignore` and only `.env.example` is committed, so no credential ever reaches the repository. Errors raised anywhere in the stack are instances of `AppError` carrying an HTTP status, caught by a single Flask error handler and rendered as a readable message.

5. Application pages

Dashboard

Four counters, the next five events and the most popular event, all computed at request time — nothing is hard-coded. The counters use `countDocuments`; the registration total unwinds the arrays and excludes cancelled entries; the popular event sorts on a confirmed count calculated with `$size` over a `$filter`. This is why the counter shows 83 while the seed reports 88: five entries are cancelled and are not campus activity.



Dashboard — counters, next five events, most popular event.

Events

The catalog with full CRUD. Search by title uses a case-insensitive regex; category, tag and past/upcoming act as additional `$match` conditions on the same pipeline; sorting flips the `startDate` direction. A red "Full" badge appears when the confirmed count reaches capacity. The create and edit form writes the embedded `location` object and the `tags` array from a comma-separated input.

Campus Event Manager						
Dashboard Events Users Analytics						
Events						
Search by title All categories All tags Past and upcoming Date, oldest first Apply Reset						
Title	Category	Starts	Location	Capacity	Confirmed	Actions
Intro to MongoDB Aggregation	Workshop	20 May 2026, 10:00	Innovation Center - B204 - Paris	30	5	Edit Delete
Cloud Cost Optimisation Talk	Talk	06 Jun 2026, 10:00	Main Building - A101 - Paris	60	4	Edit Delete
Spring Design Sprint	Workshop	27 Jun 2026, 10:00	Design Lab - D12 - Paris	20	6	Edit Delete
Data Ethics Roundtable	Talk	11 Jul 2026, 10:00	Main Building - A203 - Paris	40	3	Edit Delete
Summer Hack Night	Hackathon	26 Jul 2026, 10:00	Innovation Center - B100 - Paris	50	8	Edit Delete
Alumni Networking Evening	Networking	10 Aug 2026, 10:00	Main Building - Hall - Paris	80	5	Edit Delete
Wellbeing Yoga Session	Social	19 Aug 2026, 10:00	Sports Center - S01 - Paris	25	0	Edit Delete
Linux Fundamentals Lab	Workshop	28 Aug 2026, 10:00	Computer Lab - C05 - Saclay	24	4	Edit Delete
Kickoff: Autumn Semester	Talk	06 Sep 2026, 10:00	Main Building - Hall - Paris	100	6	Edit Delete
NoSQL Modeling Deep Dive	Workshop	13 Sep 2026, 10:00	Innovation Center - B204 - Paris	25	5	Edit Delete
Cybersecurity Capture the Flag	Hackathon	18 Sep 2026, 10:00	Computer Lab - C11 - Saclay	40	7	Edit Delete

Events — catalog with search, category, tag, period filters and date sort.

Event detail and registrations

One event with its embedded location, its organiser resolved by `$lookup`, its capacity, confirmed count and occupancy percentage, and its participant list. Registering picks an existing user and pushes an entry into the array; duplicates are rejected, and a confirmed registration is refused once the event is full, with the waiting list offered instead. Cancelling sets the embedded status through the positional operator; removing deletes the entry with `$pull`. The list re-reads the document and reflects the change immediately.

Campus Event ManagerDashboardEventsUsersAnalytics

Intro to MongoDB Aggregation

Edit

Intro to MongoDB Aggregation - organised by the campus community. Open to all students and staff.

CategoryWorkshop

Tagsnosqldatabasespython

Starts20 May 2026, 10:00

Ends20 May 2026, 13:00

LocationInnovation Center, room B204, Paris campus

OrganiserClaire Dubois · Data Engineering

Capacity30

Confirmed5

Occupancy16.7%

Register a participant

Karim Benali (karim.benali@campus.edu)

Confirmed

Register

Participants (6)

Name	Email	Status	Registered	Actions
------	-------	--------	------------	---------

Event detail — embedded location, organiser, occupancy and registration management.

Users

A searchable directory showing name, email, department, role, interests and a registration count obtained by looking up the events whose `registrations.userId` contains the user. Clicking a user opens their full history with a status per event and upcoming/past counters. Users with no registration still appear in the list and still have a valid detail page.

Campus Event Manager							
Dashboard Events Users Analytics							
Users							
New user							
Search by name or email All departments All roles Apply Reset							
Name	Email	Department	Role	Interests	Registrations		
Karim Benali	karim.benali@campus.edu	Data Engineering	student	cloud devops python	7	Edit	
Mei Chen	mei.chen@campus.edu	Data Engineering	student	statistics visualisation	5	Edit	
Rafael Costa	rafael.costa@campus.edu	Design	student	illustration ux	6	Edit	
Ines Diallo	ines.diallo@campus.edu	Business	student	finance public speaking	6	Edit	
Claire Dubois	claire.dubois@campus.edu	Data Engineering	teacher	noSQL distributed systems	5	Edit	
Lucas Fernandez	lucas.fernandez@campus.edu	Computer Science	student	cybersecurity networks	7	Edit	
Nadia Haddad	nadia.haddad@campus.edu	Business	teacher	strategy entrepreneurship	6	Edit	
Erik Johansson	erik.johansson@campus.edu	Student Services	staff	community wellbeing	6	Edit	
Jonas Lindqvist	jonas.lindqvist@campus.edu	Computer Science	student	open source linux	6	Edit	
Sofia Marchetti	sofia.marchetti@campus.edu	Business	student	entrepreneurship marketing	7	Edit	
Laura Moreau	laura.moreau@campus.edu	Student Services	staff	events logistics	4	Edit	

Users — directory with search, department and role filters, and registration counts.

User deletion has an explicit reference strategy rather than being silently allowed. If the user organises an event, deletion is blocked and the message says so. If the user only has registrations, deletion is blocked by default and a second action removes the registrations first, in one `$pull` across all events, before deleting the account.

Analytics

Six analyses, each produced by its own pipeline in `analytics_repo.py`: registrations by category, top five events by confirmed registrations, users with no registration, events above the overall average occupancy, most used tags, and events grouped by month.

Campus Event Manager

Dashboard

Events

Users

Analytics

Analytics

Every table on this page is produced by a MongoDB aggregation pipeline (`src/repositories/analytics_repo.py`).

A. Registrations by category

Category	Events	Confirmed registrations
Hackathon	3	24
Workshop	5	24
Talk	4	15
Networking	2	8
Conference	2	6
Social	2	0

B. Top 5 most popular events

Event	Category	Capacity	Confirmed	Occupancy
Autumn Hackathon 48h	Hackathon	60	9	15.0%
Summer Hack Night	Hackathon	50	8	16.0%

Analytics — Analysis A (registrations by category) and Analysis B (top five events).

6. Aggregation focus

Analysis D — events above the overall average occupancy

This is the most interesting pipeline because it needs two levels of aggregation at once: a value per event, and a value across all events, then a comparison between them. MongoDB has no window function here, so the whole collection is collapsed into a single document that holds both the average and the list of events, and is then expanded again.

```
[
  { $addFields: { confirmedCount: { $size: { $filter: {
    input: { $ifNull: ["$registrations", []] },
    as: "r", cond: { $eq: ["$r.status", "confirmed"] } } } } } },

  { $addFields: { occupancy: { $round: [
    { $multiply: [ { $divide: ["$confirmedCount",
      { $max: ["$capacity", 1] } ] }, 100 ] }, 1 ] } } },

  { $group: { _id: null,
    averageOccupancy: { $avg: "$occupancy" },
    events: { $push: { title: "$title", category: "$category",
      capacity: "$capacity", confirmedCount: "$confirmedCount",
      occupancy: "$occupancy" } } } },
```

```

{ $unwind: "$events" },
{ $match: { $expr: { $gt: ["$events.occupancy", "$averageOccupancy"] } } },
{ $project: { _id: 0, averageOccupancy: { $round: ["$averageOccupancy", 1] },
           title: "$events.title", occupancy: "$events.occupancy", /* ... */ } },
{ $sort: { occupancy: -1 } }
]

```

Three details are worth calling out. `$group` with `_id: null` is what makes the average global rather than per-category. `$push` carries every event through that bottleneck so the rows survive to be filtered afterwards — without it, the averaging stage would destroy the data it is meant to filter. And `$match` must use `$expr`, because it compares two fields of the same document rather than a field against a constant. `$max: ["$capacity", 1]` guards against a division by zero that the validator should already prevent.

Analysis C — users with no registration

The naive approach is to load every event, collect the user ids in the application and subtract. That moves the whole collection over the wire to answer a question the database can answer itself. The pipeline instead uses a `$lookup` with a sub-pipeline, which is the anti-join pattern:

```

[
  { $lookup: {
    from: "events",
    let: { uid: "$_id" },
    pipeline: [
      { $match: { $expr: { $in: ["$uid",
                               { $ifNull: ["$registrations.userId", []] } ] } } },
      { $limit: 1 },
      { $project: { _id: 1 } }
    ],
    as: "registeredEvents" } },
  { $match: { registeredEvents: { $size: 0 } } },
  { $project: { firstName: 1, lastName: 1, email: 1, department: 1, role: 1 } },
  { $sort: { lastName: 1 } }
]

```

`$registrations.userId` resolves to the array of all user ids inside the event, so `$in` tests membership directly against the embedded array — no `$unwind` needed. `$limit: 1` inside the sub-pipeline is the point of the design: the question is "does any event exist?", not "how many?", so the lookup stops at the first match instead of building a full list per user. The empty-array test then keeps exactly the users nothing matched, and the sub-pipeline's `$project` keeps the joined payload down to an `_id`.

7. Conclusion

Difficulties

The hardest part was not the query language but the modeling judgement: deciding where the document boundary should sit. The first instinct was a separate `registrations` collection, which is the relational reflex and would have thrown away the main advantage of a document store. Reasoning about how the data is actually read — always per event, always bounded by capacity — is what settled it.

On the aggregation side, Analysis D took the longest. The first version grouped the events away and then had nothing left to filter; understanding that `$group` plus `$push` followed by `$unwind` is the standard way to compare a row against a global value was the unlock. Operationally, the Atlas cluster had to be abandoned because the network blocked port 27017, which is why the project ships against a local MongoDB server.

Limitations

- No authentication: any visitor can register anyone, which is acceptable for the brief but not for a real deployment.
- Capacity is enforced by a read-then-write sequence. Under real concurrency two simultaneous registrations could both pass the check; a production version would push the capacity test into the update filter itself or use a transaction.
- Denormalised venue names would need an `updateMany` if a building were renamed.
- No pagination: fine for 18 events, not for 18,000.

Possible improvements

- Make the waiting list active: automatically promote the first waiting entry when a confirmed registration is cancelled.
- Add a text index on title and description for real search instead of a regex scan.
- Expose the analytics as a small JSON API and draw charts from it.
- Add a test suite against a disposable database, seeded by the same script.